PROMPT для GOOGLE AI STUDIO / GEMINI FLASH 3.8

STORKCRYPTO: ПОКРОКОВЕ ЗАВЕРШЕННЯ PRODUCTION, STARS, TON CONNECT І БЕЗПЕКИ

Скопіюй цей документ у Google AI Studio для Gemini Flash 3.8.

Роль і правила роботи

Працюй як Senior Full-Stack Engineer, Cloudflare Engineer, Telegram Payments Engineer, TON Connect Security Engineer і Chief Design Developer.

Проєкт:

```
Plain Text
```

```
https://storkcrypto.pages.dev/
```

Технології:

```
Plain Text
```

```
React
Zustand
Cloudflare Pages/Functions
Supabase
Telegram Mini App
Telegram Bot API
Telegram Stars XTR
TON Connect
```

Перед змінами:

1. Проаналізуй весь repository.
2. Перевір актуальну Telegram і TON документацію.
3. Не використовуй застарілу схему GRAM як синонім Telegram Stars.
4. Не вигадуй API endpoints, яких немає в коді.

5. Не став `READY`, якщо не пройдено реальний acceptance test.
6. Не виводь secrets у звіт або logs.
7. Працюй спочатку на test environment.
8. Не списуй реальні Stars без окремого підтвердження власника.

ЧАСТИНА A. Рішення щодо валют

A1. Telegram Stars

Для PRO digital subscription у Telegram використовуй:

```
Plain Text
currency = XTR
```

Обов’язкова схема:

```
Plain Text
Telegram.WebApp.initData
→ server-side HMAC validation
→ create XTR invoice
→ Telegram.WebApp.openInvoice(invoiceUrl)
→ pre_checkout_query
→ answerPreCheckoutQuery within 10 seconds
→ successful_payment
→ verify payload, user, amount, currency, charge id
→ activate PRO server-side
```

`pre_checkout_query` не є доказом платежу. PRO можна активувати тільки після `successful_payment`.

Для digital goods `provider_token` може бути порожнім.

A2. Gram

Розділяй поняття:

```
Plain Text
```

```
Telegram Stars = XTR, Telegram payment system
Gram = native TON blockchain currency
```

Не використовуй Gram у Telegram Stars invoice.

Не додавай кнопку Gram для тієї самої PRO digital subscription у Telegram Mini App, якщо немає окремого юридично та технічно затвердженого on-chain product flow.

Якщо repository вже містить Gram UI, зроби одне з двох:

1. повністю прибери Gram із production UI для PRO;
2. або ізолюй його як окремий feature flag `TON_GRAM_PAYMENTS_ENABLED=false` у production.

Не залишай кнопку, яка нічого не робить.

ЧАСТИНА B. Підготовка owner/developer environment

Поясни власнику, що він повинен виконати власноруч у правильній послідовності.

B1. GitHub

Bash

```bash
git status
git checkout -b fix/stars-ton-production
git rev-parse HEAD
git remote -v
```

Перед змінами збережи commit SHA.

Після змін:

Bash

```bash
git add .
git commit -m "fix: production Telegram Stars and TON Connect flows"
git push -u origin fix/stars-ton-production
```

Не пушити `.env`, bot token, Supabase service role або private keys.

B2. Cloudflare Pages

У Cloudflare Dashboard:

```
Workers & Pages
→ storkcrypto project
→ Settings
→ Variables and Secrets
```

Налаштувати secrets тільки для Functions/production environment:

```
TELEGRAM_BOT_TOKEN
TELEGRAM_BOT_USERNAME
TELEGRAM_WEBHOOK_SECRET
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
TON_API_KEY або TON_RPC_URL
TON_PAYMENT_RECEIVER_ADDRESS, якщо окремий Gram flow дозволено
APP_ORIGIN=https://storkcrypto.pages.dev
PAYMENTS_ENV=production або test
```

Не додавати secrets у Vite public variables типу:

```
VITE_TELEGRAM_BOT_TOKEN
VITE_SUPABASE_SERVICE_ROLE_KEY
VITE_TON_PRIVATE_KEY
```

Після deploy перевірити:

```
Cloudflare build: successful
Functions: deployed
Production commit SHA: expected SHA
Environment: correct
```

B3. Bot configuration

Власник повинен перевірити через BotFather і Telegram bot setup:

Plain Text

```
bot exists
Mini App URL = https://storkcrypto.pages.dev/
webhook points to deployed server endpoint
bot token belongs to the same bot
terms/support commands exist
```

Якщо webhook встановлюється через Bot API, виконати тільки з secret-safe terminal:

Bash

```
curl -sS -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/setWebhook"
  -d "url=https://storkcrypto.pages.dev/api/telegram/webhook" \
  -d "secret_token=${TELEGRAM_WEBHOOK_SECRET}" \
  -d 'allowed_updates=["message","pre_checkout_query"]'
```

Перевірити webhook:

Bash

```
curl -sS "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/getWebhookInfo"
```

Ніколи не вставляти реальний token у prompt, frontend або screenshots.

ЧАСТИНА C. Repository audit

Виконай пошук:

Bash

```
grep -RIn --exclude-dir=node_modules --exclude-dir=.git \
  -E "tg_user|SIMULATE TEST CONFIRMATION|fake.*payment|isPro.*true|GRAM|MetaMas
```

Знайди:

- yci payment endpoints;
- yci webhook handlers;
- plan definitions;
- subscription state;
- wallet components;

- manifest;
- DEV flags;
- localStorage/Zustand persisted state;
- Cloudflare functions;
- fallback HTML routing.

Зроби карту фактичних файлів перед зміною.

ЧАСТИНА D. Telegram Stars implementation

D1. Create endpoint

Перевір/створи:

```
Plain Text

/functions/api/payments/stars/create.ts
```

Вимоги:

```
Plain Text

No initData → 401 JSON
Invalid HMAC → 401 JSON
Frontend user id → ignored
Frontend amount → ignored
Frontend currency → ignored
Valid initData → authenticated Telegram user
```

Backend plan catalog:

```
Plain Text

const PLANS = {
  STARTER: { amount: 99, durationDays: 30 },
  PRO: { amount: 250, durationDays: 30 },
  WHALE: { amount: 500, durationDays: 30 },
} as const;
```

Frontend передає лише:

JSON

```
{ "planId": "PRO" }
```

Backend сам визначає amount, duration і capabilities.

Order повинен бути idempotent і мати:

Plain Text

```
orderId
telegramUserId
planId
amount = backend value
currency = XTR
invoicePayload
status = PENDING
expiresAt
```

D2. Frontend invoice

Використовуй:

Plain Text

```
const tg = window.Telegram?.WebApp;

if (!tg?.initData) {
  showTelegramOnlyMessage();
  return;
}

const response = await fetch('/api/payments/stars/create', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `tma ${tg.initData}`,
  },
  body: JSON.stringify({ planId }),
});

const data = await response.json();
if (!response.ok || !data.invoiceUrl) throw new Error(data.error);
```

```
tg.openInvoice(data.invoiceUrl, () => {
  refreshStatusFromServer(orderId);
});
```

Callback не активує PRO.

D3. Webhook

Реалізуй:

```
Plain Text

POST /api/telegram/webhook
```

Для `pre_checkout_query`:

```
Plain Text

find order by payload
validate user id
validate XTR
validate amount
validate plan
validate pending and not expired
answer within 10 seconds
```

Для `successful_payment`:

```
Plain Text

find order
validate payload
validate user
validate XTR
validate amount
validate telegram_payment_charge_id
insert payment idempotently
mark order PAID
activate subscription in one transaction
```

Для duplicate webhook:

```
Plain Text

return success without duplicating payment or subscription
```

D4. Status endpoint

Реалізуй:

```
Plain Text

GET /api/payments/stars/status?orderId=...
```

Відповіді:

```
Plain Text

No auth → 401 JSON
Foreign order → 403 JSON
Unknown order → 404 JSON
Pending → 200 PENDING
Paid and verified → 200 PAID
```

Переконайся, що Cloudflare SPA fallback не повертає `index.html` для `/api/*`.

D5. Refund/cancel/support

Реалізуй або документуй:

- refund через `refundStarPayment`;
- продовження/скасування Stars subscription через підтримуваний Bot API flow;
- Terms page;
- Support page/command;
- refund policy;
- customer payment lookup;
- admin payment audit log без секретів.

ЧАСТИНА E. Gram / TON flow — тільки окремо і лише якщо власник справді його хоче

За замовчуванням залиш:

```
Plain Text
```

```
TON_GRAM_PAYMENTS_ENABLED=false
```

Якщо Gram потрібен для окремого дозволеного product flow, реалізуй повністю:

1. Generate unique invoice ID.
2. Store invoice with user/order/amount/expiry.
3. Use TON Connect for wallet interaction.
4. Create transaction with exact recipient and amount.
5. Include invoice ID in comment or use unique deposit address.
6. Backend monitors TON transactions.
7. Verify recipient.
8. Verify exact amount.
9. Verify invoice ID/comment or address mapping.
10. Reject expired/reused invoice.
11. Wait for masterchain finality.
12. Credit order once.
13. Store transaction hash, lt, account and timestamp.
14. Add reconciliation job.
15. Add manual recovery for missing/malformed comments.
16. Never accept only frontend callback or external message lookup as payment proof.
17. Never store private key in frontend.

Не називай Gram `Telegram currency` і не передавай його у Telegram invoice `currency`.

ЧАСТИНА F. TON Connect

1. Use one root `TonConnectUIProvider`.
2. Use `/tonconnect-manifest.json`.
3. Change `iconUrl` from JPG to `/icon-180.png`.
4. Add terms/privacy URLs.
5. Ensure manifest GET works without auth/CORS challenge.
6. Add Portfolio `Connect TON Wallet`.

7. Show connected address.
8. Add Disconnect.
9. Use `useIsConnectionRestored()`.
10. Close picker after successful connection.
11. If using wallet authentication, implement `ton_proof` with:

```
server nonce
short expiration
single use
expected domain
network validation
signature verification
address derivation
atomic consume
```

Wallet connection does not activate PRO.

Do not add MetaMask or Ethereum wallet protocol to this Telegram Mini App. Follow TON Connect and Telegram blockchain guidelines.

ЧАСТИНА G. Три плани

Створи UX із чотирма рівнями:

```
BASIC — free
STARTER — 99 XTR / 30 days
PRO — 250 XTR / 30 days, Popular
WHALE — 500 XTR / 30 days
```

BASIC — dashboard, demo portfolio, limited AI and Academy.

STARTER — limited signals, 10–20 AI scans/day, basic alerts and Academy.

PRO — expanded signals, AI Market Scan, Whale Radar, Heatmap, no ads, full Academy, advanced alerts.

WHALE — all PRO, VIP Whale Radar, advanced on-chain data, custom alerts, priority AI, early access.

Показуй плани в одній зрозумілій modal. Не ховай реальні обмеження. Не називай план VIP, якщо він не має відмінних функцій.

ЧАСТИНА H. Інші обов’язкові виправлення

Academy

UA/EN/RU мають змінювати весь текст card, tags, lessons, video title, quiz, CTA і errors.

Market data

Уніфікуй:

```
Plain Text

LIVE / DELAYED / STALE / DEMO / ERROR
```

Не показуй LIVE для cached data.

Modals

```
Plain Text

informational/technical → fullscreen
PRO subscription → compact exception
```

Проведи 30 циклів кожного modal type.

Loading/design

Збережи vertical 6,5-second Manifesto.

```
Plain Text

rotating concentric rings
→ fog
→ StorkCrypto
→ fracture
→ clear pixels
```

```
→ horizontal data convergence
→ dashboard
```

Не використовуй `GENESIS`, semicircle disappearance або строкатий multicolor design. Збільш читабельність тексту мінімально. `StorkCrypto` залиш компактним і яскравим.

ЧАСТИНА I. Команди перевірки для власника

Після deploy власник може виконати безпечні read-only перевірки:

Bash

```bash
curl -i https://storkcrypto.pages.dev/tonconnect-manifest.json
curl -I https://storkcrypto.pages.dev/icon-180.png
curl -i -X POST https://storkcrypto.pages.dev/api/validate \
  -H 'content-type: application/json' \
  --data '{}'

curl -i -X POST https://storkcrypto.pages.dev/api/payments/stars/create \
  -H 'content-type: application/json' \
  --data '{"planId":"PRO"}'

curl -i 'https://storkcrypto.pages.dev/api/payments/stars/status?orderId=TEST'
```

Expected:

Plain Text

```
manifest → 200 JSON
icon → 200 image/png
validate without initData → 400
stars create without initData → 401
stars status without auth → 401
```

Не запускай реальний payment test через curl. Реальна Stars payment можлива тільки в Telegram-контексті.

ЧАСТИНА J. Acceptance tests

Stars

Plain Text

```
No auth = 401
Fake auth = 401
Valid Telegram initData = invoice
Invoice opens in Telegram
Cancel = BASIC
pre_checkout_query received and answered <=10s
successful_payment received
Before successful_payment = no PRO
After successful_payment = PRO
Duplicate webhook = no duplicate
Refund path documented/tested
```

Gram, якщо feature disabled

Plain Text

```
No Gram button in production UI
No /api/payments/gram/create expectation
No Gram as XTR
```

TON Wallet

Plain Text

```
Manifest 200
PNG icon 200
Connect picker
Picker closes
Connected address shown
Disconnect
30 connect/disconnect cycles
Optional ton_proof verified server-side
```

Application

Plain Text

```
AI Chat 10 requests
Day/Night
Academy UA/EN/RU content
```

```
Terminal assets
News
Loading timing 6.5s ±100ms
Modal 30 cycles
Mobile 360x800 and 390x844
CSP and secrets scan
No uncaught console errors
```

Фінальний звіт

Поверни:

Plain Text

```
1. Current commit SHA
2. Production Cloudflare deployment SHA
3. Changed files
4. Functions deployed
5. Environment variables names configured
6. Stars tests
7. Webhook tests
8. Refund/support readiness
9. Gram status: disabled or fully implemented
10. TON Connect tests
11. ton_proof tests
12. Three plans status
13. Academy status
14. Modal status
15. Loading status
16. Security scan
17. Remaining risks
18. READY / NOT READY
```

Заборонено писати `READY`, якщо:

Plain Text

```
successful_payment не підтверджено
PRO можна активувати frontend-only
DEV simulation у production
status endpoint повертає HTML
TON wallet picker не закривається
Gram button веде в нікуди
```

```
Academy toggle не змінює контент
orphan modal overlays > 0
```

Джерела, які потрібно використовувати при перевірці

- https://core.telegram.org/bots/payments-stars
- https://core.telegram.org/bots/api
- https://core.telegram.org/bots/webapps
- https://core.telegram.org/bots/blockchain-guidelines
- https://docs.ton.org/applications/ton-connect/get-started
- https://docs.ton.org/applications/ton-connect/how-to/ton-proof
- https://docs.ton.org/applications/payments/overview
- https://docs.ton.org/applications/payments/gram